

JAVA PROGRAMMING – EXPLAINED NOTES

Unit-wise theory + practical examples based on the provided syllabus

UNIT I – Basic Syntactical Constructs in Java

This unit establishes the object-oriented foundation of Java. The important idea is that Java programs are organized around classes and objects, and the language provides features such as encapsulation, method overloading, arrays, strings and access control.

1. Java Platform and Program Structure

Java source code is compiled into bytecode. The Java Virtual Machine (JVM) executes this bytecode, which is why the same compiled program can run on different operating systems when a compatible JVM is available. The JDK provides development tools, the JRE provides the runtime environment, and the JVM performs bytecode execution.

```
class Demo {  
    void show() {  
        System.out.println("Java program");  
    }  
}
```

2. Class, Object and Members

A class is a blueprint containing data and behavior. An object is a runtime instance of a class. Methods define behavior, while fields represent the state of an object. Creating objects with new allocates an object and invokes its constructor.

```
class Student {  
    String name;  
  
    void display() {  
        System.out.println(name);  
    }  
}  
  
Student s = new Student();  
s.name = "Amit";  
s.display();
```

3. Tokens and Expressions

Java programs are formed from tokens such as keywords, identifiers, literals, operators and separators. Expressions combine values and operators to produce a result. Operators include arithmetic, relational, logical, assignment, unary, conditional and bitwise operators. Type casting converts a value from one type to another when the conversion is permitted.

```
int a = 10;  
double b = 3.5;  
double result = a + b;  
  
int x = (int) b;          // explicit casting  
boolean ok = a > 5 && b < 5;
```

4. Decision and Looping

Control flow determines which statements execute and how many times they execute. if/else is used for conditions, switch is useful for multiple fixed choices, and for/while/do-while are used for repetition. break terminates a loop or switch, while continue skips the current iteration.

```

for (int i = 1; i <= 5; i++) {
    if (i == 3)
        continue;
    System.out.println(i);
}

```

5. Arrays and String Classes

An array stores a fixed number of elements of the same type. String objects are immutable, meaning their contents cannot be changed after creation. StringBuilder/StringBuffer are used when repeated modifications are required; StringBuffer provides synchronized methods and is therefore useful in thread-safe contexts where required.

```

int[] marks = {70, 82, 91};

String name = "Java";
StringBuffer sb = new StringBuffer("Core");
sb.append(" Java");

```

6. Constructors and Methods

A constructor initializes an object and has the same name as the class. It has no return type. Constructors can be overloaded. Method overloading means multiple methods have the same name but different parameter lists. Java does not support user-defined operator overloading.

```

class Account {
    private int balance;

    Account() {
        balance = 0;
    }

    Account(int balance) {
        this.balance = balance;
    }

    void deposit(int amount) {
        balance += amount;
    }
}

```

7. Access Control and Garbage Collection

public members are accessible broadly, private members are restricted to their class, protected members are available to the class, package and suitable subclasses, and default/package-private members are accessible within the same package. Java automatically manages unused objects through garbage collection; programmers normally do not manually free object memory.

```

class SecureData {
    private int value;

    public void setValue(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
    }
}

```

UNIT II – Inheritance, Interfaces and Packages

This unit focuses on reuse and abstraction. Inheritance allows a class to reuse behavior from another class, interfaces define contracts, and packages organize related classes.

1. Inheritance

Inheritance creates an IS-A relationship. A subclass can reuse accessible members of a superclass and add or redefine behavior. Common forms discussed in Java are single, multilevel and hierarchical inheritance. Java avoids multiple inheritance of classes, but multiple interfaces can be implemented.

```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

```

2. Method Overriding and super

Overriding occurs when a subclass supplies its own implementation of an inherited method with the same signature. Runtime polymorphism allows the overridden implementation to be selected through a superclass reference. super can access superclass members or invoke its constructor.

```

class Animal {
    void sound() {
        System.out.println("Sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

```

3. final and Abstract Classes

A final variable cannot be reassigned after initialization, a final method cannot be overridden, and a final class cannot be extended. An abstract class cannot normally be instantiated and may contain abstract methods that subclasses must implement.

```

abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }
}

```

4. Interfaces

An interface defines a contract that implementing classes must follow. A class can implement multiple interfaces. Interface-based design reduces coupling because code can depend on a capability rather than a specific implementation.

```

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Report");
    }
}

```

5. Packages

A package groups related classes and helps avoid naming conflicts and control access. Classes can be imported from another package. User-defined packages are commonly used to organize larger applications into logical modules.

```

package college.notes;

public class Utility {

```

```

        public static void show() {
            System.out.println("Utility");
        }
    }

    // Another file:
    // import college.notes.Utility;

```

UNIT III – Exception Handling and Multithreading

This unit covers two major reliability concepts: handling abnormal conditions without abruptly terminating normal application flow, and executing independent tasks concurrently.

1. Errors vs Exceptions

Errors generally represent serious problems that applications are not expected to recover from. Exceptions represent conditions that application code can often handle. Checked exceptions are checked by the compiler; unchecked exceptions are generally subclasses of RuntimeException.

```

try {
    int result = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Invalid arithmetic operation");
}

```

2. try, catch, finally, throw and throws

try contains risky code, catch handles a matching exception, and finally is commonly used for cleanup that should happen regardless of success or failure. throw explicitly creates/raises an exception, while throws declares that a method may pass a checked exception to its caller.

```

static void validate(int age) {
    if (age < 18)
        throw new IllegalArgumentException("Age not valid");
}

try {
    validate(15);
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
}

```

3. User-defined Exceptions

A custom exception gives domain-specific meaning to an error condition. It can extend Exception for a checked exception or RuntimeException for an unchecked exception.

```

class InvalidMarksException extends Exception {
    InvalidMarksException(String message) {
        super(message);
    }
}

```

4. Thread Creation and Life Cycle

A thread is a lightweight unit of execution. A thread can be created by extending Thread or, more flexibly, by implementing Runnable. Typical states include new, runnable, running, blocked/waiting and terminated. The JVM scheduler decides when runnable threads receive CPU time.

```

class Task implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}

Thread t = new Thread(new Task());
t.start();

```

5. Synchronization

When multiple threads access shared mutable state, operations can interfere with each other. synchronized methods/blocks provide mutual exclusion so that only one thread at a time enters the protected section for the same lock.

```
class Counter {
    private int count;

    synchronized void increment() {
        count++;
    }
}
```

UNIT IV – AWT, Swing and Event Handling

This unit introduces Java desktop GUI development. AWT provides heavyweight components backed by native system peers, while Swing provides a richer set of mostly lightweight components.

1. AWT Component Hierarchy

A component is a visible GUI element. Containers hold components. Frame represents a top-level window, while Panel groups components. Common controls include Label, Button, Checkbox, TextField and TextArea.

```
import java.awt.*;

class MyFrame extends Frame {
    MyFrame() {
        add(new Label("Name"));
        add(new TextField(20));
        setSize(400, 200);
        setVisible(true);
    }
}
```

2. Layout Managers

Layout managers control how components are positioned without hard-coding coordinates. FlowLayout places components sequentially, BorderLayout divides a container into regions, GridLayout creates equal cells, and GridBagLayout supports more flexible arrangements.

```
setLayout(new BorderLayout());
add(new Button("North"), BorderLayout.NORTH);
add(new Button("Center"), BorderLayout.CENTER);
```

3. Swing

Swing is part of Java's GUI toolkit and provides components such as JFrame, JLabel, JTextField, JButton, JCheckBox, JRadioButton, JComboBox, JTable, JTree, JTabbedPane and JScrollPane. Swing applications generally use event-driven programming.

```
import javax.swing.*;

JFrame frame = new JFrame("Demo");
JButton button = new JButton("Save");
frame.add(button);
frame.setSize(300, 200);
frame.setVisible(true);
```

4. Delegation Event Model

Java uses the delegation event model. An event source generates an event, and a registered listener receives it. For example, a JButton can generate an ActionEvent handled by an ActionListener. This separates event generation from event-processing code.

```
button.addActionListener(e -> {
    System.out.println("Button clicked");
});
```

5. Important Event Listeners

ActionListener handles actions such as button clicks; ItemListener handles item-state changes; KeyListener handles keyboard events; MouseListener handles mouse actions; MouseMotionListener handles mouse movement; TextListener handles text changes in AWT text components.

UNIT V – Basics of Network Programming

Java networking APIs allow programs to communicate across machines using addresses, sockets, TCP/UDP and URL-based resources.

1. Client–Server Model

In the client-server model, a server waits for requests and a client initiates communication. A socket represents one endpoint of a network connection. TCP provides reliable, ordered, connection-oriented communication, while UDP is connectionless and does not guarantee delivery or order.

```
// Server concept:
ServerSocket server = new ServerSocket(5000);
Socket socket = server.accept();

// Client concept:
Socket socket = new Socket("localhost", 5000);
```

2. InetAddress

InetAddress represents an IP address and provides methods for resolving host names. It is useful when an application needs information about the network address associated with a host name.

```
InetAddress address =
    InetAddress.getByName("example.com");

System.out.println(address.getHostAddress());
```

3. TCP/IP Sockets

TCP communication commonly uses ServerSocket on the server side and Socket on both sides. After a connection is established, input and output streams can carry data. Resources should be closed properly, preferably using try-with-resources.

```
try (ServerSocket server = new ServerSocket(5000);
     Socket socket = server.accept()) {

    // communicate through socket streams
}
```

4. Datagram and URL

DatagramSocket and DatagramPacket are used for UDP communication. URL represents a web/resource address, while URLConnection provides a connection to the resource and access to metadata or content streams.

```
URL url = new URL("https://example.com");
URLConnection connection = url.openConnection();
System.out.println(connection.getContentType());
```

UNIT VI – Interacting with Database using JDBC

JDBC (Java Database Connectivity) is the standard Java API for interacting with relational databases. A typical application establishes a connection, prepares SQL, executes it, processes results and closes resources.

1. JDBC Architecture

In a two-tier model, the Java application communicates more directly with the database through a JDBC driver. In a three-tier model, the client communicates with an application/business layer, which communicates with the database. The three-tier approach is common in larger systems because it separates presentation, business logic and data access.

2. JDBC Drivers

JDBC drivers translate JDBC calls into database-specific communication. The syllabus covers the driver concept and driver types. Modern JDBC applications commonly rely on drivers that are discovered through the JDBC driver mechanism rather than manually loading a driver class.

3. Core JDBC Interfaces

DriverManager can obtain database connections. Connection represents an active database session. Statement executes SQL, PreparedStatement supports parameterized SQL and is preferred for user-supplied values, while ResultSet represents rows returned by a query.

```
String sql = "SELECT name FROM students WHERE id = ?";

try (Connection con = DriverManager.getConnection(
    url, user, password);
    PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setInt(1, 101);

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("name"));
        }
    }
}
```

4. Statement vs PreparedStatement

Statement is suitable for simple fixed SQL, but PreparedStatement is generally safer and more reusable when values are supplied at runtime. Parameter binding avoids constructing SQL by string concatenation and helps prevent SQL injection.

```
PreparedStatement ps =
    con.prepareStatement(
        "UPDATE students SET name=? WHERE id=?");

ps.setString(1, "Rahul");
ps.setInt(2, 101);
ps.executeUpdate();
```

5. Practical JDBC Flow

A clean JDBC operation follows this sequence: obtain connection → create Statement/PreparedStatement → bind parameters if required → execute query/update → process ResultSet when applicable → close resources. try-with-resources is recommended because it automatically closes JDBC resources.

Quick Revision – Core Java Concepts

- Class = blueprint; Object = instance of a class.
- Encapsulation = keeping data controlled through class boundaries and methods.
- Inheritance = reusing/extending behavior from another class.
- Polymorphism = one interface/reference, multiple possible implementations.
- Abstraction = exposing essential behavior while hiding implementation details.
- Interface = contract that implementing classes agree to follow.
- Exception handling = controlled handling of exceptional runtime conditions.
- Thread = independent unit of execution within a process.
- Socket = endpoint used for network communication.
- JDBC = Java API used to communicate with relational databases.

Important practical focus: For programming practice, concentrate on OOP design, inheritance and overriding, interfaces, exception handling, threads and synchronization, event-driven GUI programming, socket communication, and JDBC CRUD operations.